Section: Sec 1. Introduction To Python

Lecture: 7. Programming Paradigms

Section 1: Lecture Summary

This lecture introduces the concept of **programming paradigms**, which are methodologies or approaches used in software development to organize and write code effectively. It explains that Python supports multiple paradigms: **procedural programming, object-oriented programming, modular programming, and functional programming**. The instructor provides an overview of each paradigm, emphasizing their roles in managing complexity in larger programs. Procedural programming involves breaking a program into smaller functions, making debugging and development easier. Modular programming extends this by grouping functions into interchangeable modules. Object-oriented programming bundles data and functions into classes and objects, allowing a more natural and organized design. Finally, functional programming is briefly described as supporting mathematical functions, which is beneficial for data science and AI tasks with Python.

Section 2: Key Concepts and Explanations

- **Programming Paradigms**

Programming paradigms are comprehensive strategies or methodologies for structuring software applications. Different languages support different paradigms. Python supports multiple paradigms, making it flexible.

- **Procedural Programming**

- Involves writing a sequence of instructions or code lines to perform tasks.

- Monolithic programs: large single-block programs where debugging is difficult.

- Solution: break the program into smaller pieces called **functions or procedures**, each performing a minor or complete task.

- Benefits: Easier debugging, focused development on parts, supports team collaboration by dividing functions among developers.

- Function = group of instructions performing a specific task.

- Example visualization: main program calls many smaller functions.

- **Modular Programming**

- Extension of procedural programming.

- Groups related functions into **modules or libraries**.

- Key advantage: modules can be replaced or upgraded independently without affecting the entire program.

- Analogous to replacing components in engineering devices.

- Modules contain functions, procedures (also called routines), objects, and variables.

- **Object-Oriented Programming (OOP)**

- Focuses on combining data and the operations on that data together.

- Introduces **classes** as blueprints bundling variables (data) and functions (methods) together.

- Objects are instances of classes, containing both data and behavior.

- Benefits: Encapsulation, easier management of related data and functions, reusable blueprints.

- Example: A class `Account` with data members (variables) and methods (functions operating on those variables).

- Python is naturally object-oriented – everything is an object and built-in classes are extensively used.

- **Functional Programming**

- Inspired by mathematical functions where output depends solely on input (`f(x)`).

- Promotes writing functions with no side effects or mutable state.

- Python supports functional programming concepts and provides many built-in mathematical/statistical functions.

- Important for data science, machine learning, and AI applications.

- **Relationship Between Paradigms in Python**

- Python strongly supports **procedural programming**, making it easy to write functions and work step-by-step.

- It fully supports **object-oriented programming**, where everything can be treated as objects of classes.

- It supports **modular programming**, encouraging breaking down large programs into independent modules.

- It supports **functional programming** principles that aid in mathematical and statistical computations.

Section 3: Example Code and Use Cases

Example sketch of procedural programming: using functions to break the program

perform small task 1

perform small task 2

main program calls various functions

```
def task1():
    print("Task 1 completed")

def task2():
    print("Task 2 completed")

def main_program():
    task1()
    task2()

main_program()
```

Explanation: This example shows how a large program can be broken down into smaller functions called separately to organize code clearly.

Rough sketch of an Object-Oriented Programming example - Class and Object

Creating object of class Account

```
class Account:
    def __init__(self, owner, balance=0):
        self.owner = owner  # data variable
        self.balance = balance  # data variable

    def deposit(self, amount):
        self.balance += amount  # operation on data
        print(f"{amount} deposited. New balance: {self.balance}")

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            print(f"{amount} withdrawn. New balance: {self.balance}")
        else:
            print("Insufficient funds")

acc = Account("John Doe", 1000)
acc.deposit(500)
acc.withdraw(200)
```

Explanation: This shows how data (owner, balance) and operations (deposit, withdraw) are bundled in a class, creating an object to use those functions and variables.

Section 4: Key Takeaways

- Programming paradigms are essential methodologies for structuring and developing software efficiently.

- Python supports four major paradigms: procedural, modular, object-oriented, and functional.

- Procedural programming involves breaking a large program into smaller functions for manageability.

- Modular programming groups functions into independent modules that can be updated or replaced easily.

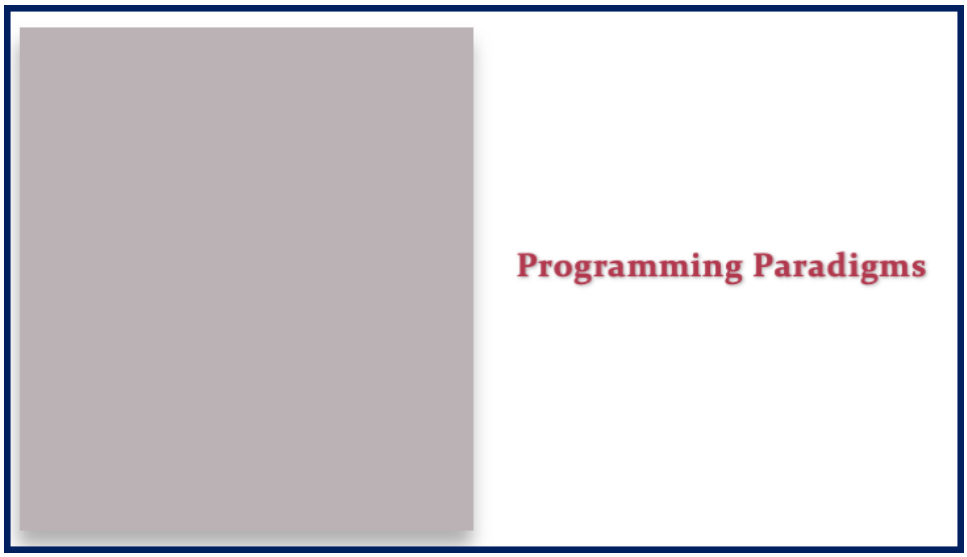
- Object-oriented programming combines data and functions in classes and uses objects for real-world modeling and reuse.

- Functional programming in Python leverages mathematical functions — important for statistics and AI-related fields.

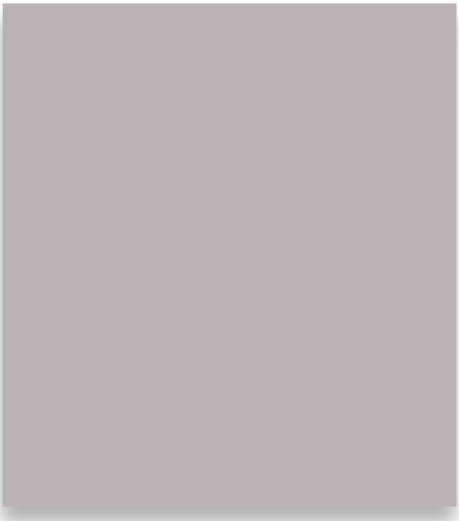
- Understanding these paradigms helps manage complexity and collaborate effectively when developing large Python applications.

Lecture in Slides

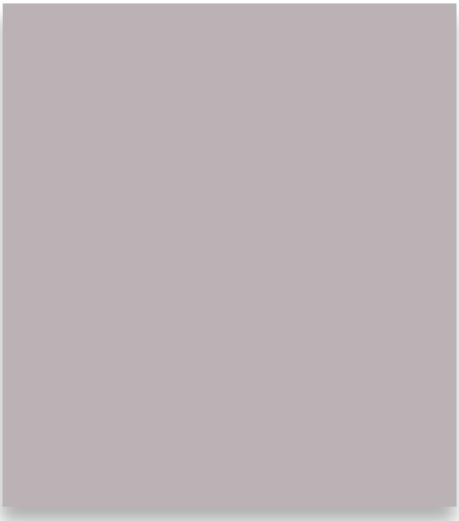
Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.



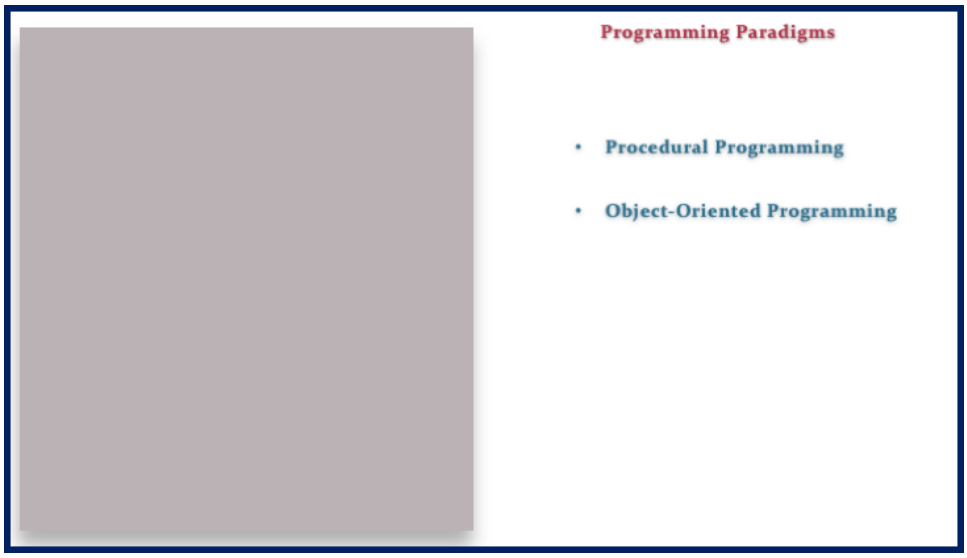
Programming Paradigms

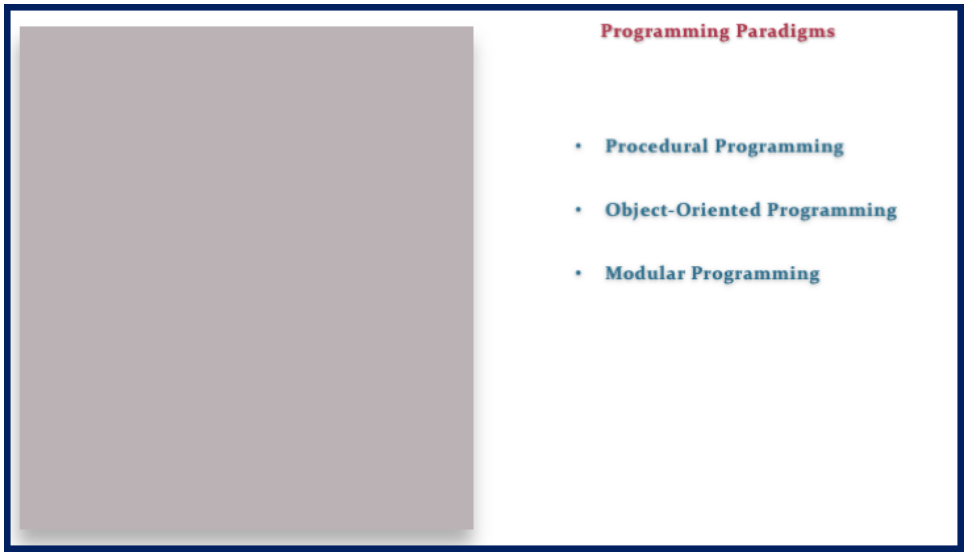


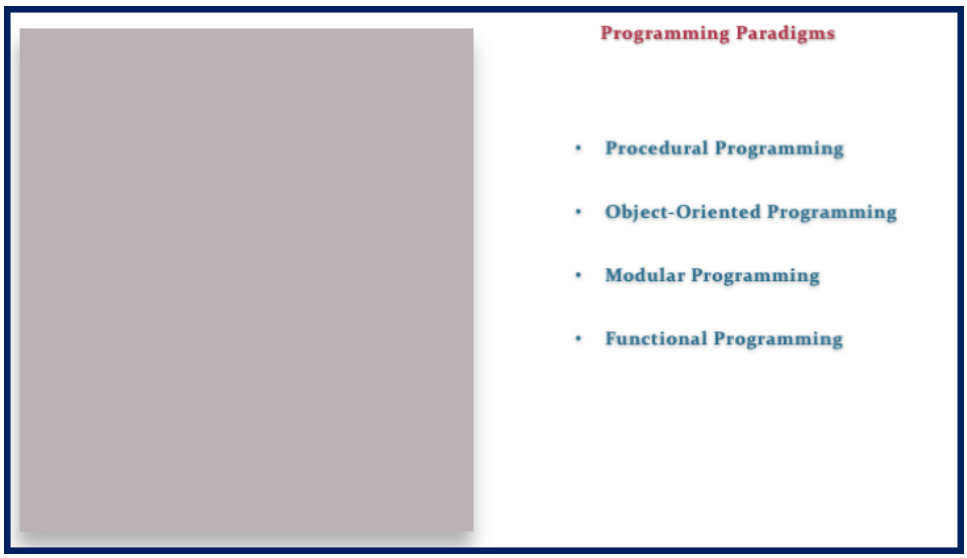
Programming Paradigms

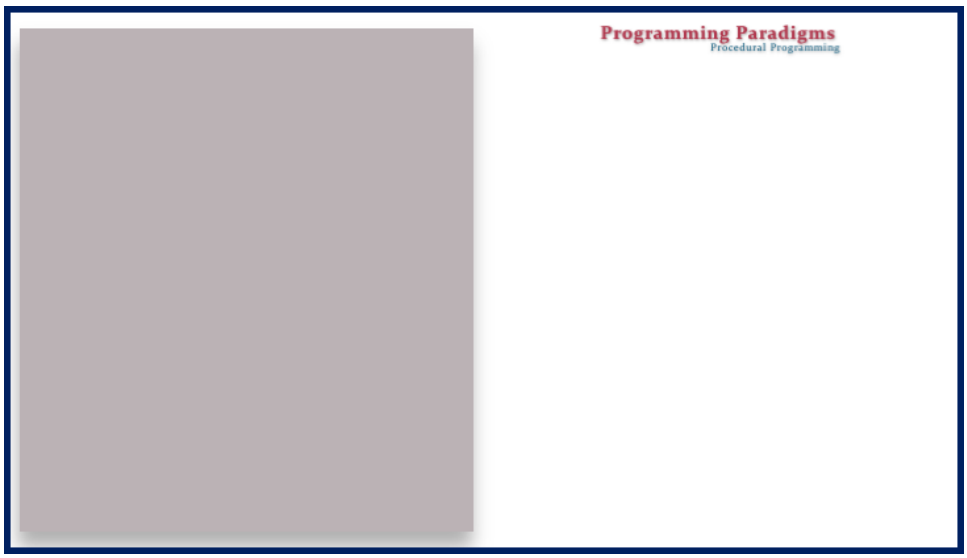


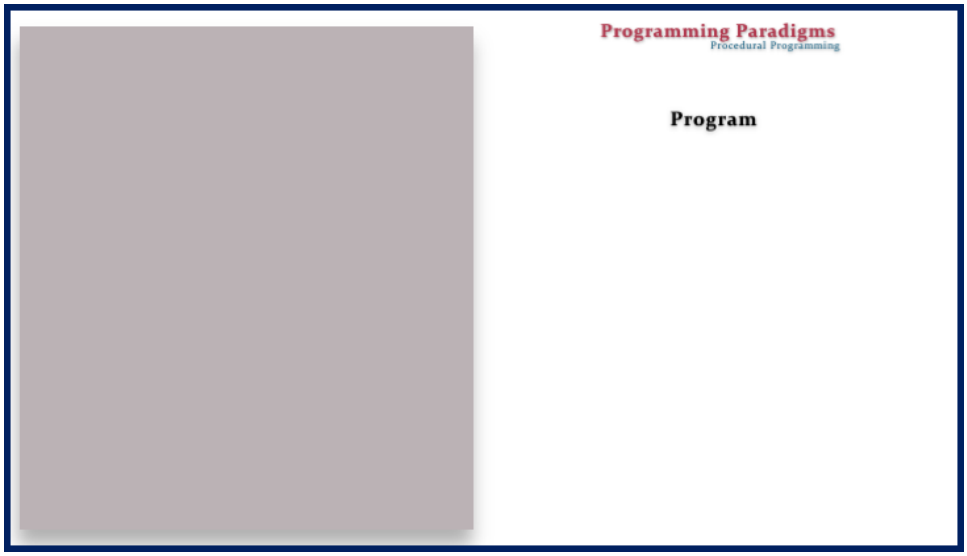
- Procedural Programming

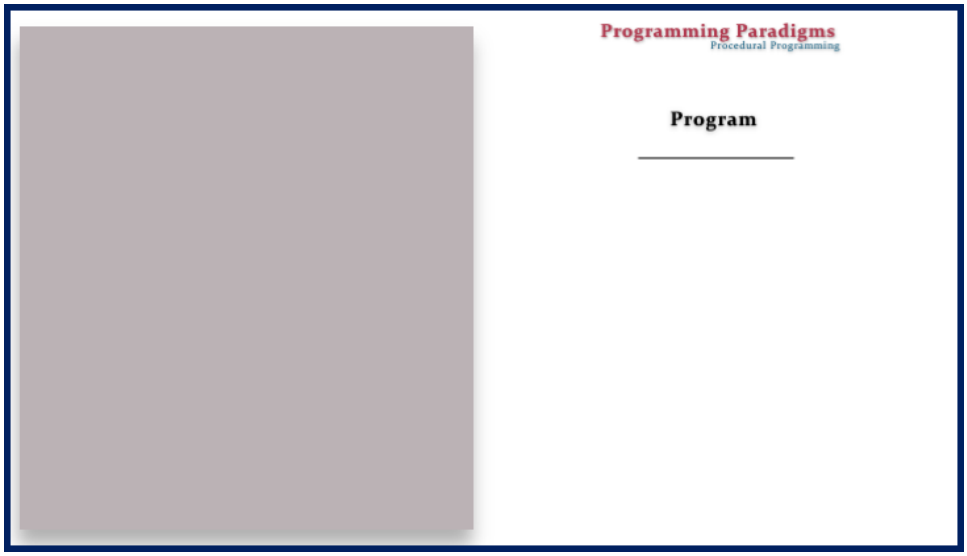

Programming Paradigms
• Procedural Programming
• Object-Oriented Programming

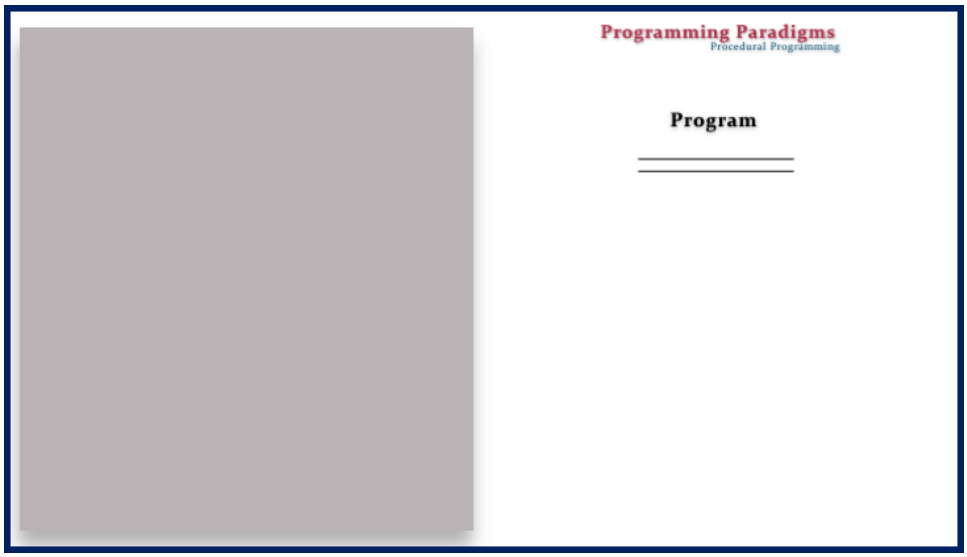

Programming Paradigms
• Procedural Programming
• Object-Oriented Programming
• Modular Programming

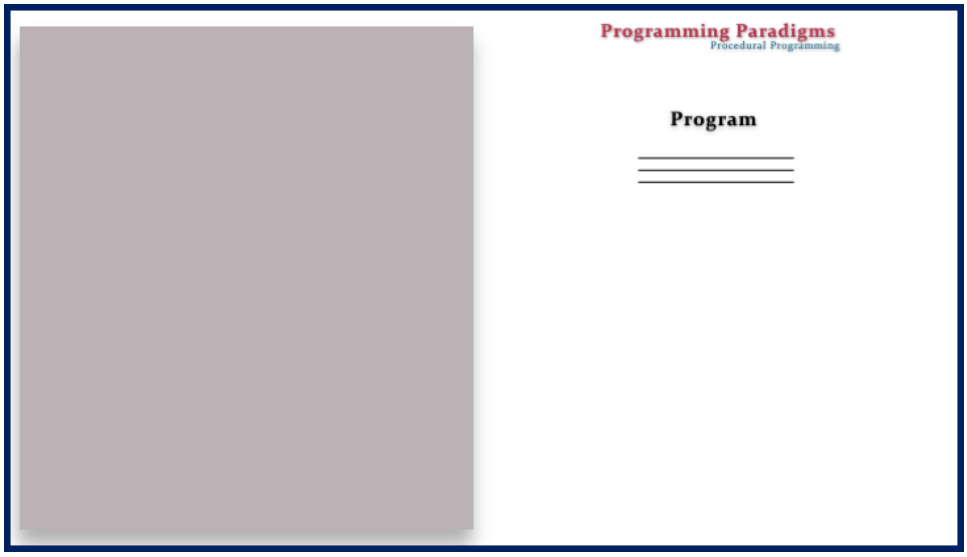

Programming Paradigms
Procedural Programming
Object-Oriented Programming
Modular Programming
Functional Programming

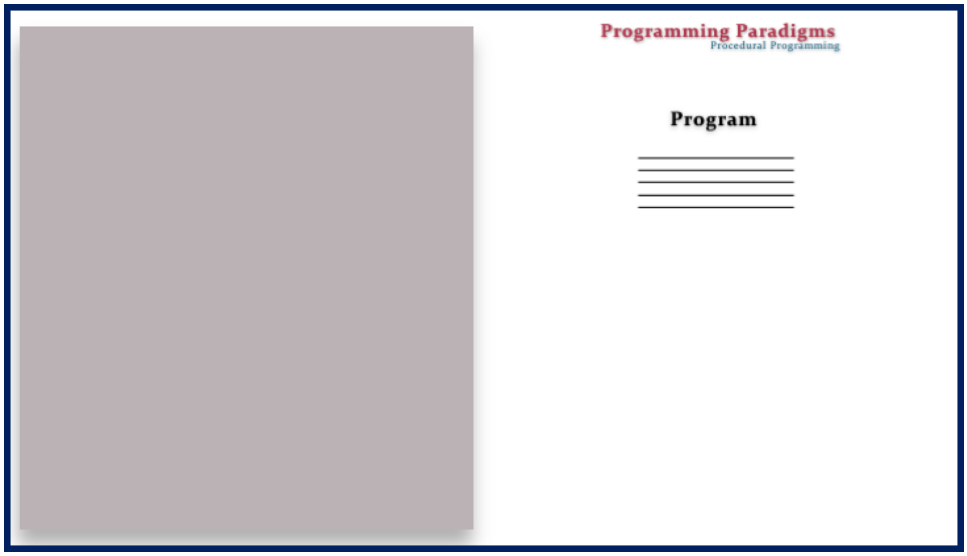

Programming Paradigms
Procedural Programming

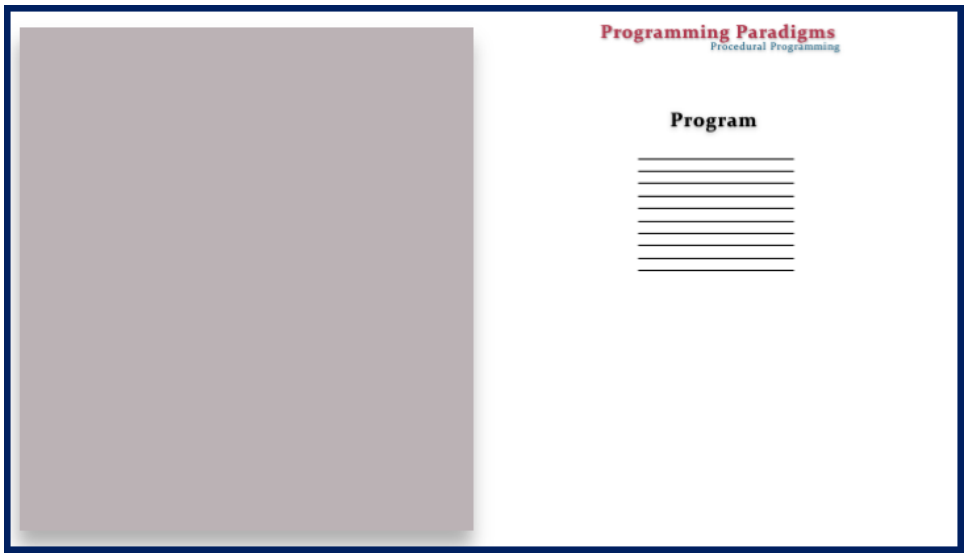

Programming Paradigms
Procedural Programming
Program

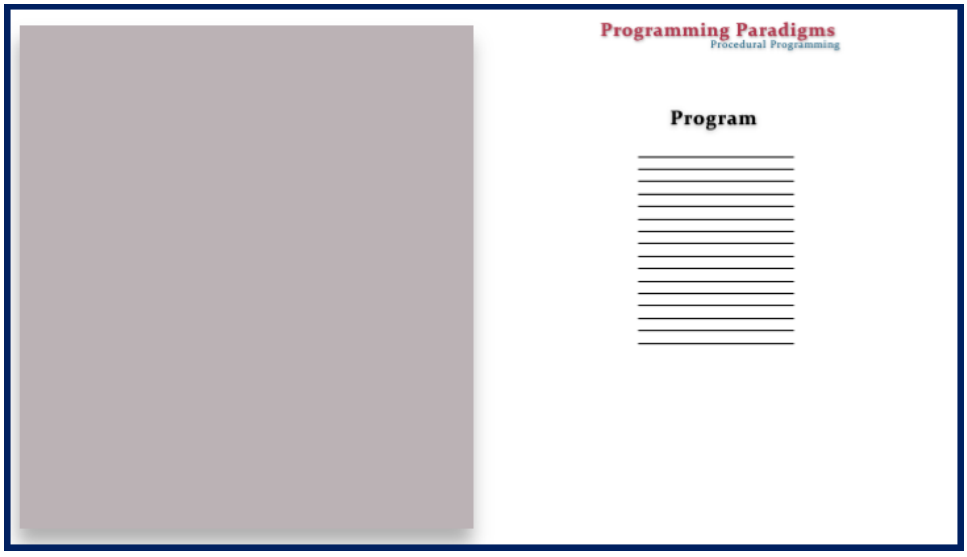

Programming Paradigms
Procedural Programming
Program

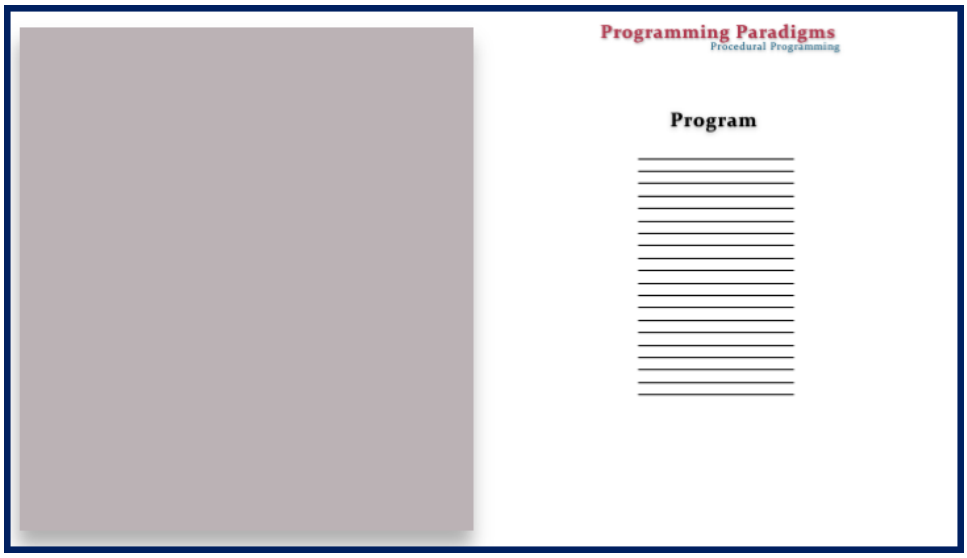

Programming Paradigms
Procedural Programming
Program

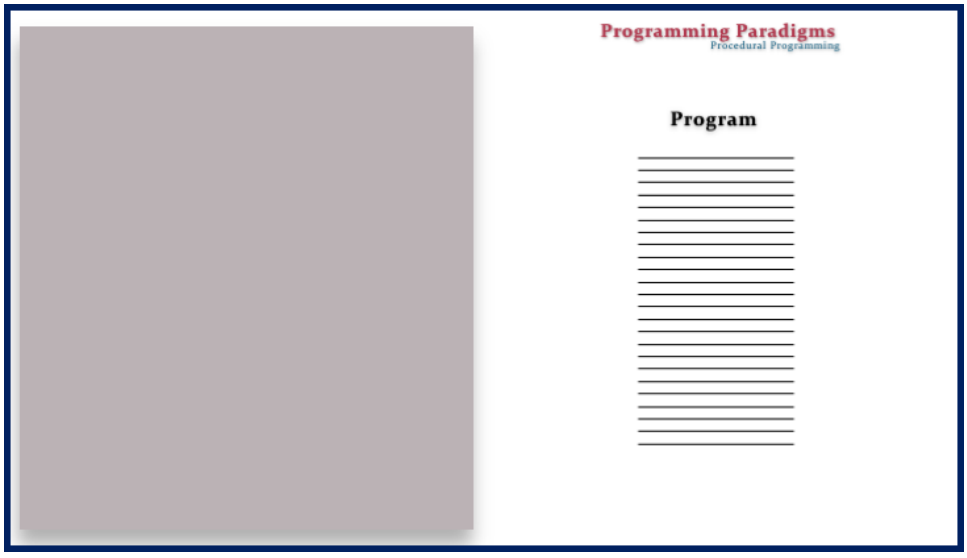

Programming Paradigms
Procedural Programming
Program

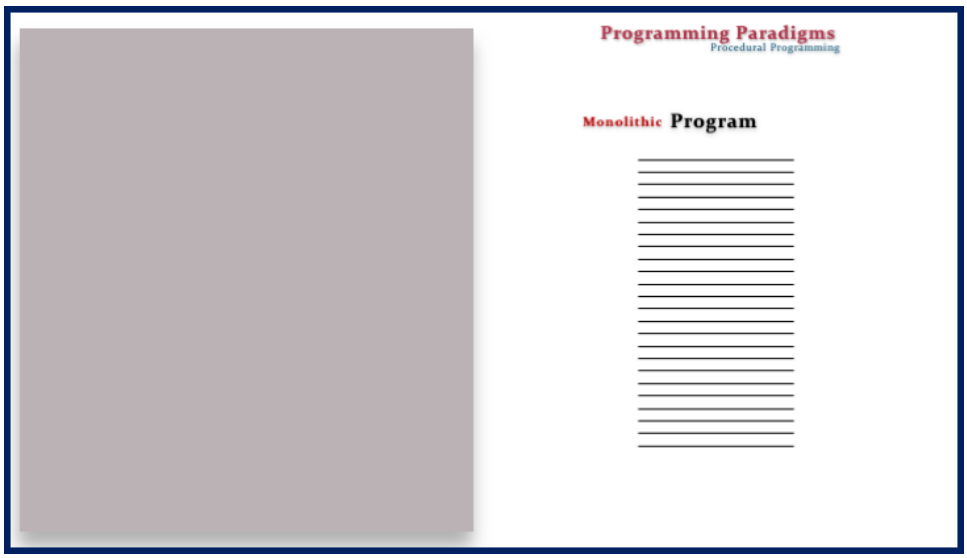

Programming Paradigms
Procedural Programming
Program

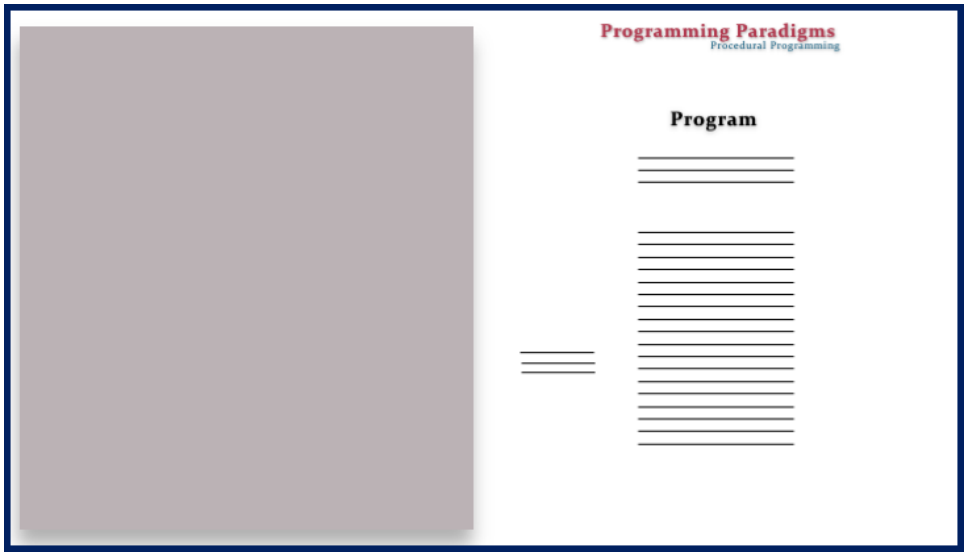

Programming Paradigms
Procedural Programming
Program

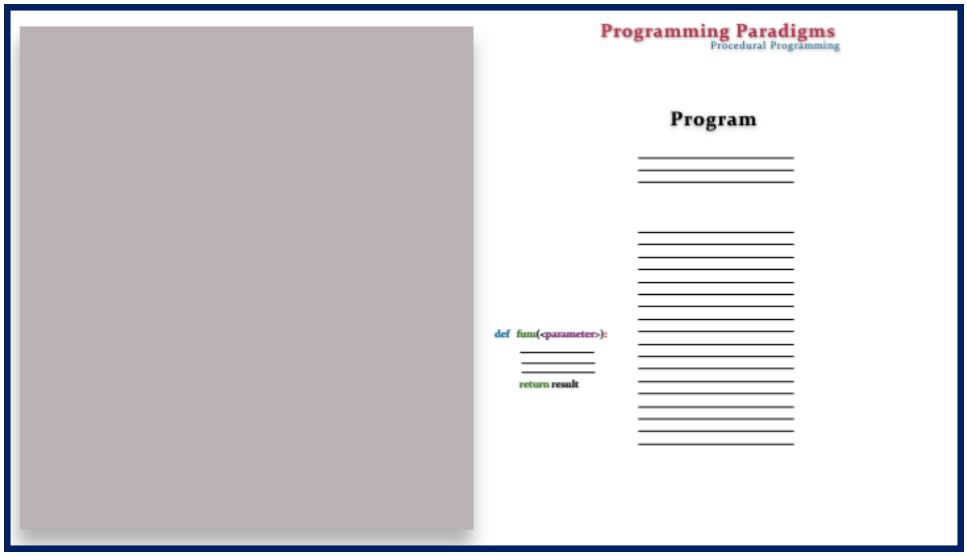

Programming Paradigms
Procedural Programming
Program

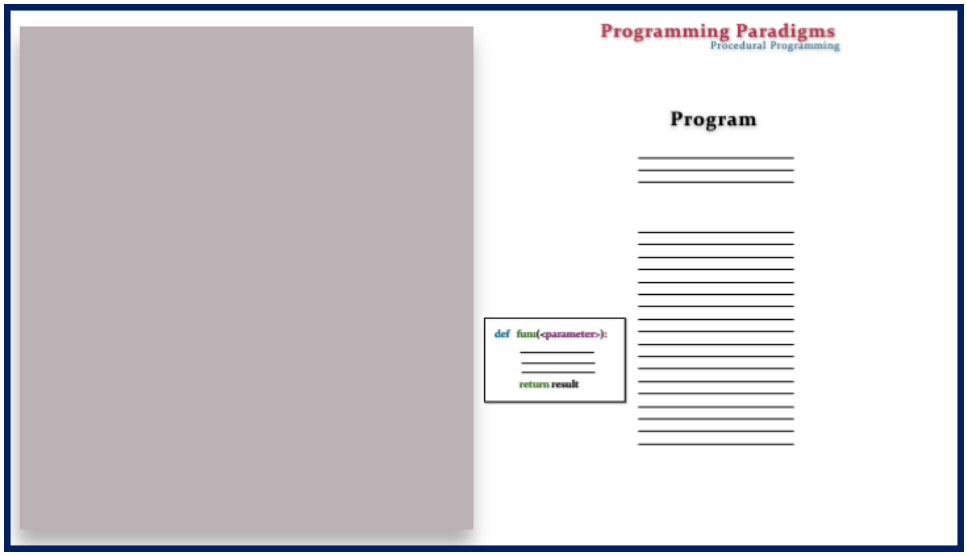

Programming Paradigms
Procedural Programming
Program

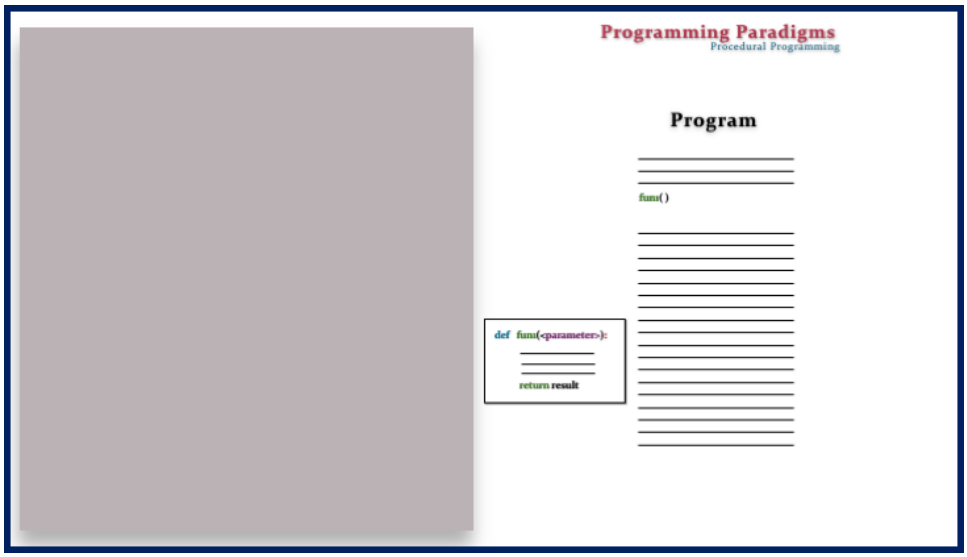

Programming Paradigms
Procedural Programming
Program

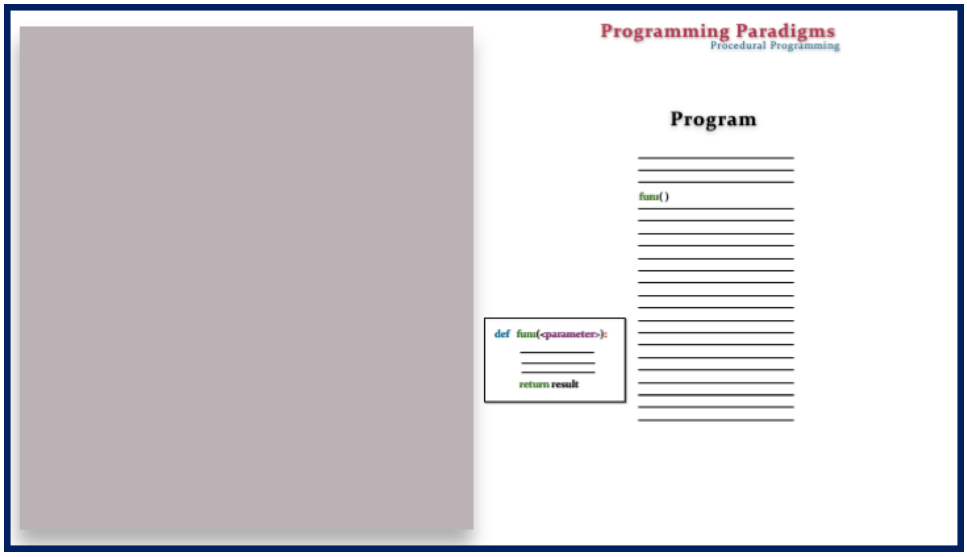

Programming Paradigms
Procedural Programming
Monolithic Program

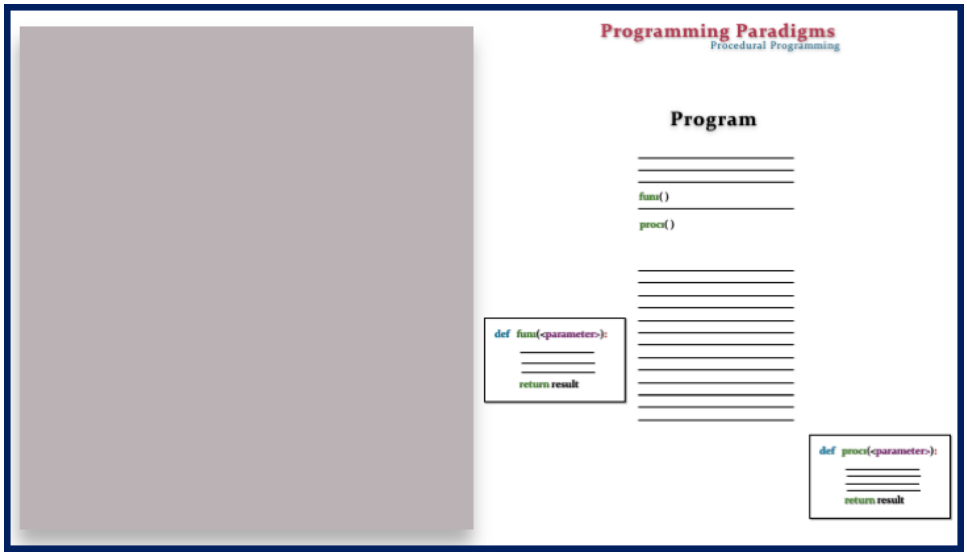

Programming Paradigms
Procedural Programming
Program

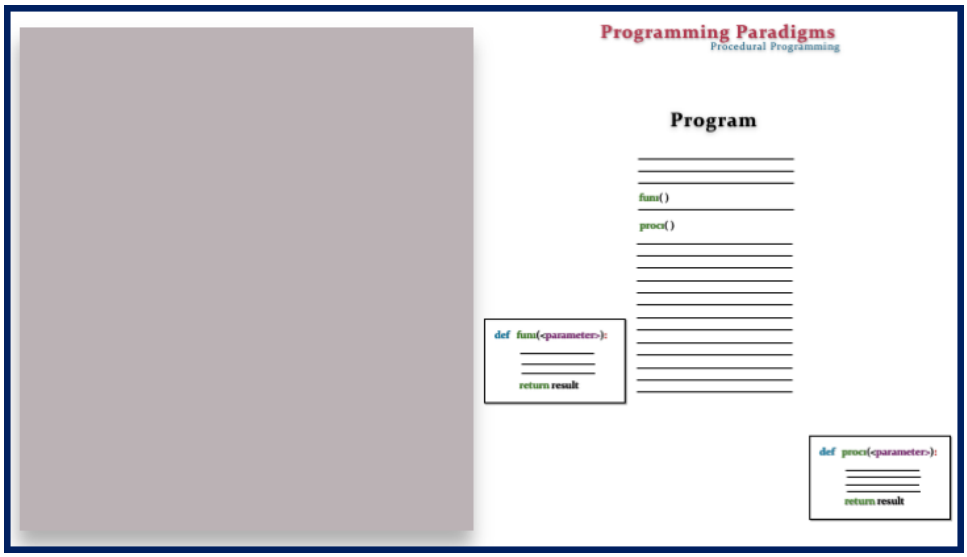

Programming Paradigms
Procedural Programming
Program
def funt(<parameter>):
return result

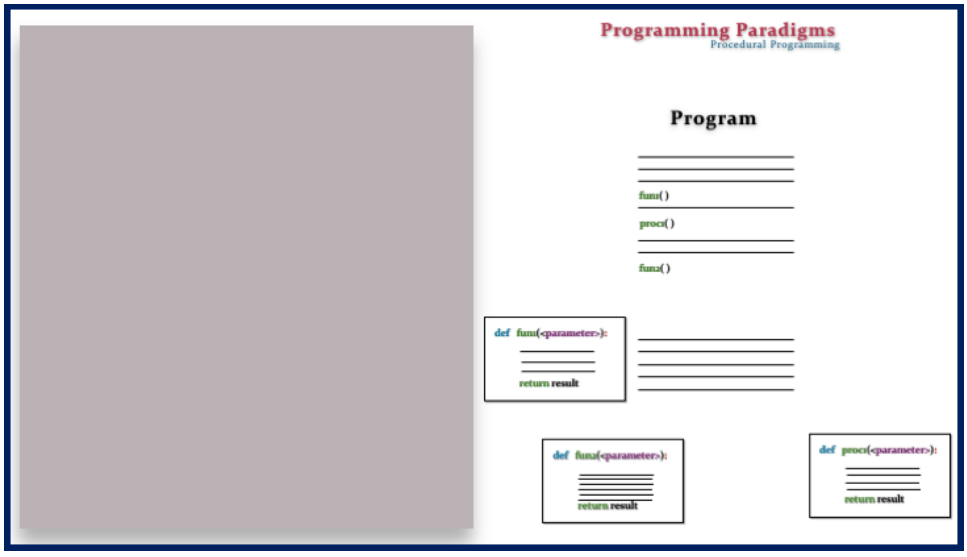

Programming Paradigms
Procedural Programming
Program
def funt(<parameter>):
return result

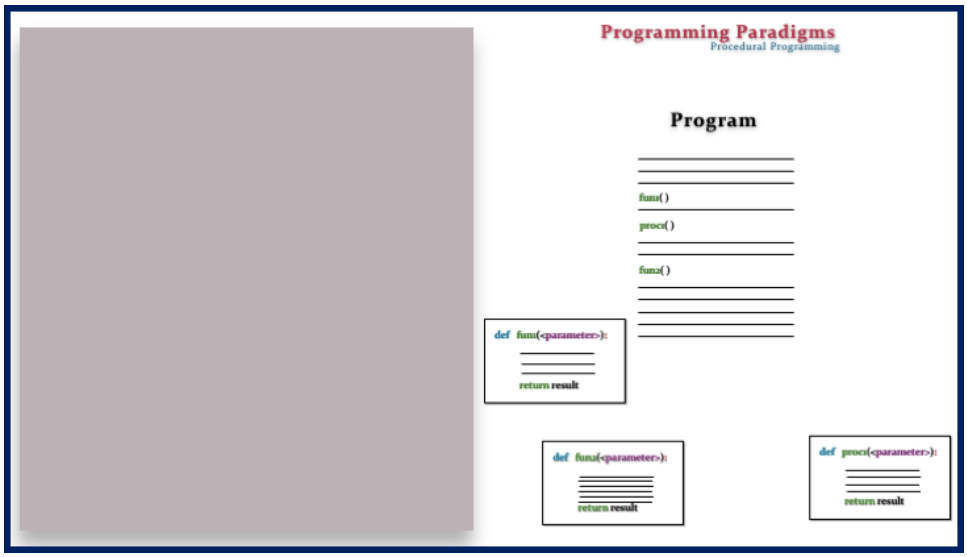

Programming Paradigms
Procedural Programming
Program
funu( )
def funu(<parameter>):
return result

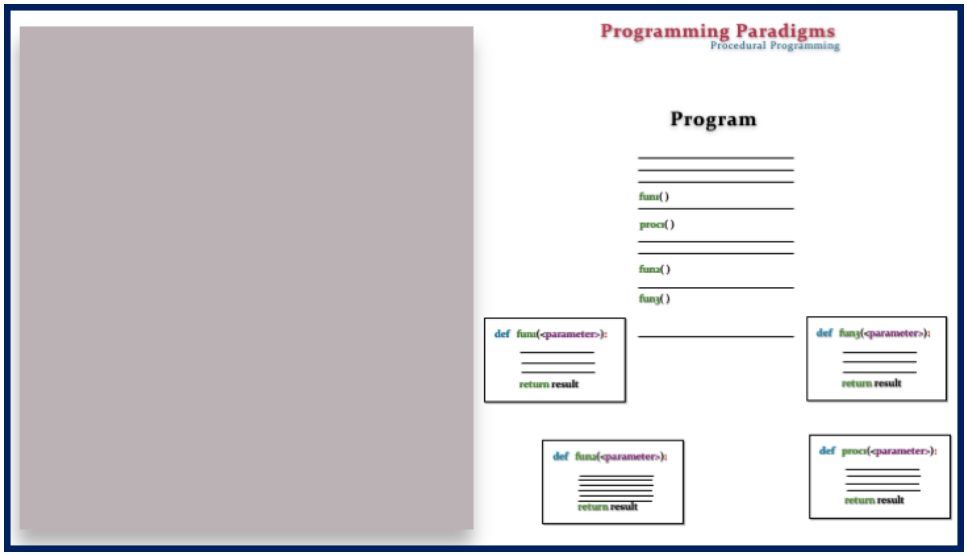

Programming Paradigms
Procedural Programming
Program
funu( )
def funu(<parameter>):
return result

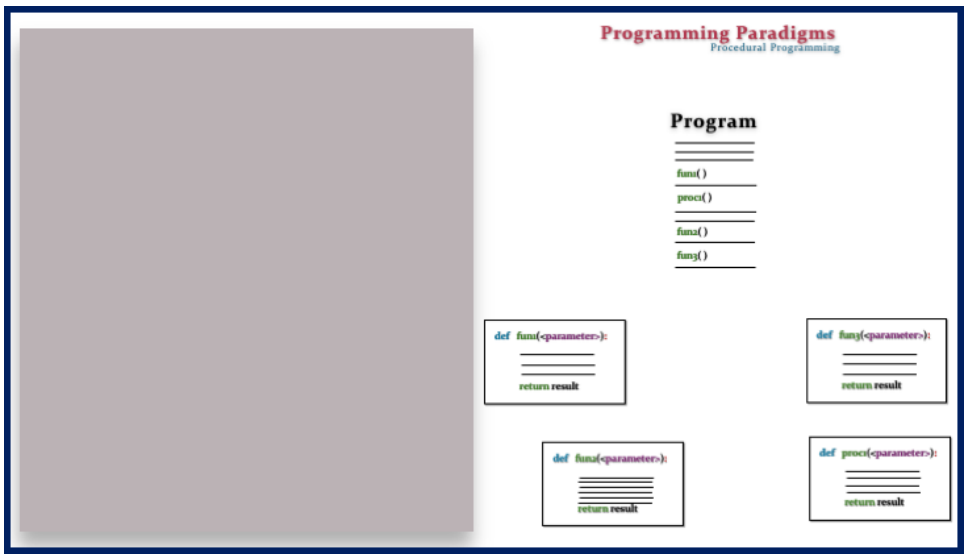

Programming Paradigms
Procedural Programming
Program
func( )
procs( )
def func(<parameter>):
return result
def procs(<parameter>):
return result

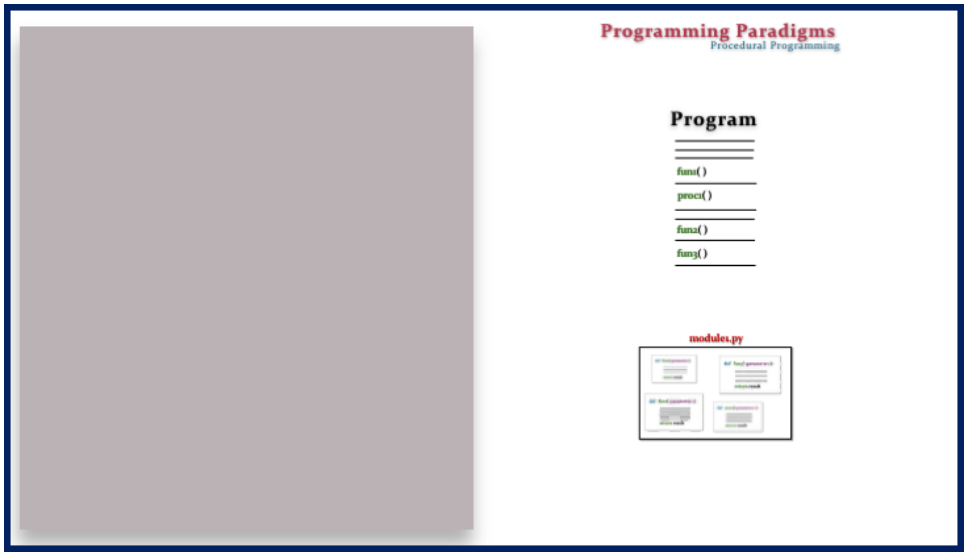

Programming Paradigms
Procedural Programming
Program
func1( )
proc1( )
def func1(<parameter>):
return result
def proc1(<parameter>):
return result

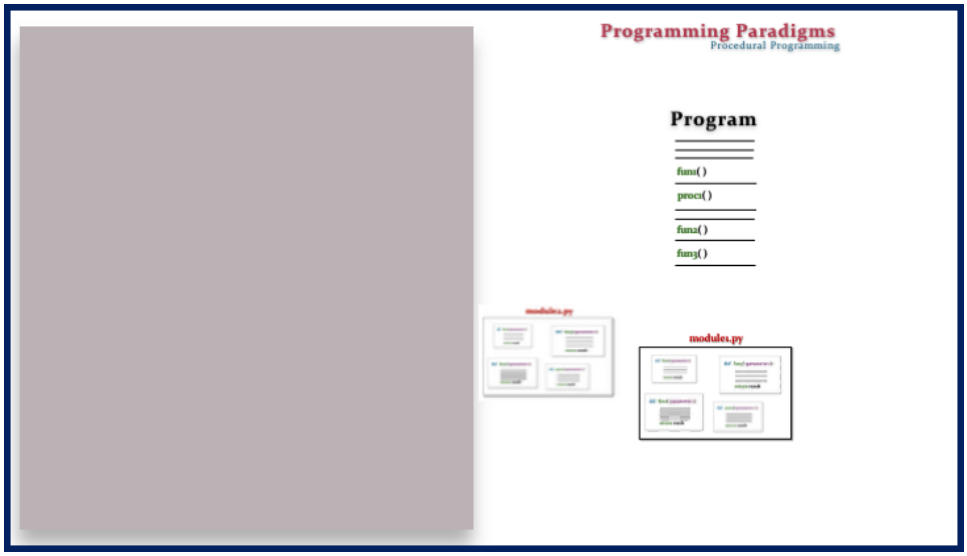

Programming Paradigms
Procedural Programming
Program
func1( )
proc1( )
func2( )
def func1(<parameter>):
return result
def func2(<parameter>):
return result
def proc1(<parameter>):
return result

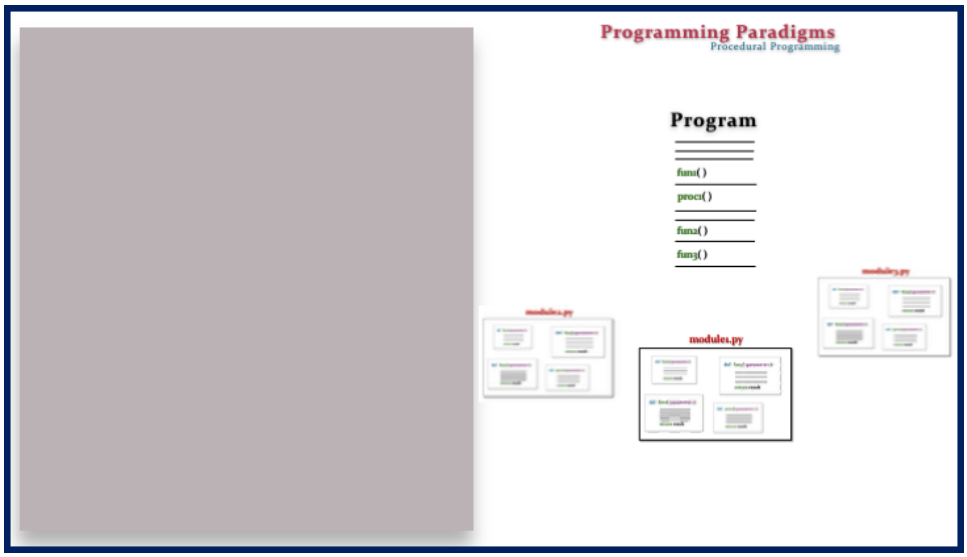

Programming Paradigms
Procedural Programming
Program
fun1( )
proc1( )
fun2( )
def fun1(<parameter>):
return result
def fun2(<parameter>):
return result
def proc1(<parameter>):
return result

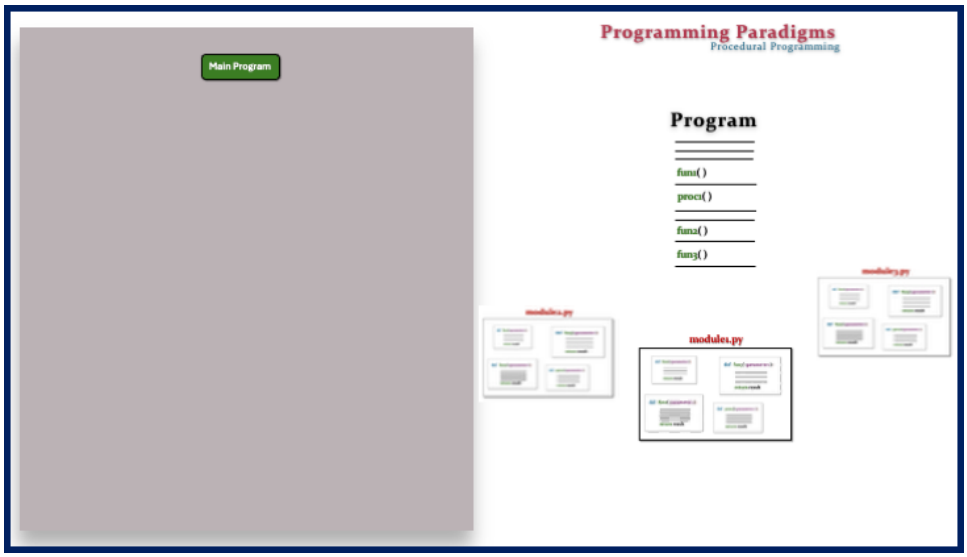

Programming Paradigms
Procedural Programming
Program
fun1( )
proc1( )
fun2( )
fun3( )
def fun1(<parameter>):
return result
def fun3(<parameter>):
return result
def fun2(<parameter>):
return result
def proc1(<parameter>):
return result

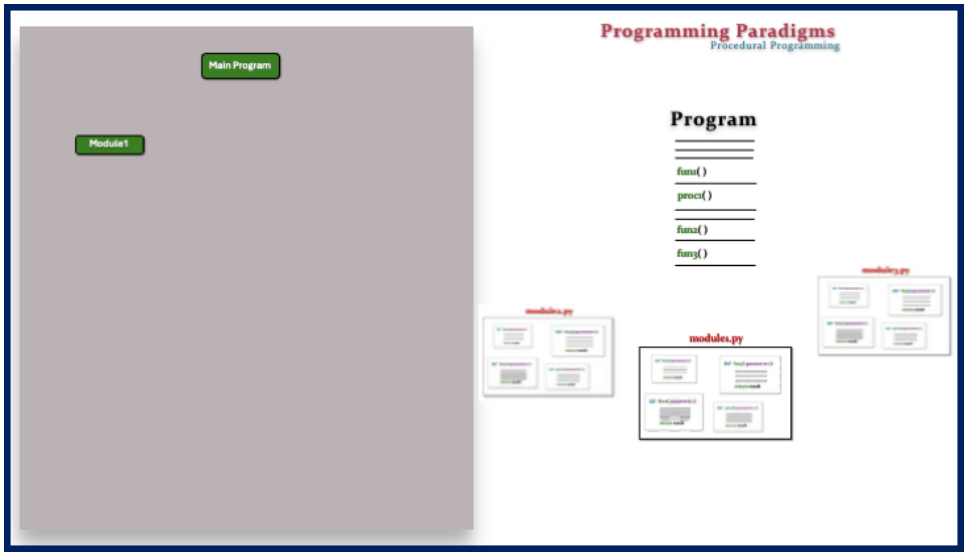

Programming Paradigms
Procedural Programming
Program
fun1( )
proc1( )
fun2( )
fun3( )
def fun1(<parameter>):
return result
def fun3(<parameter>):
return result
def fun2(<parameter>):
return result
def proc1(<parameter>):
return result

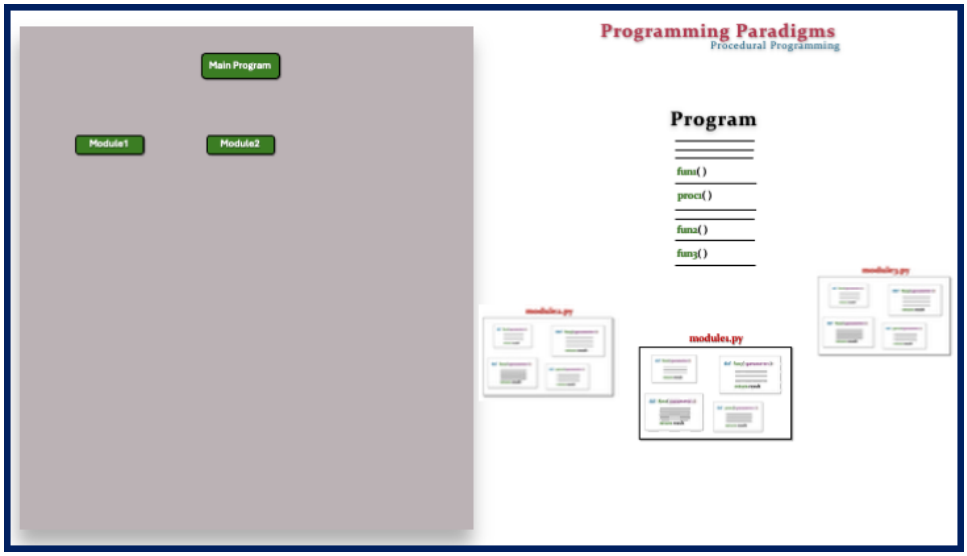

Programming Paradigms
Procedural Programming
Program
fun1( )
proc1( )
fun2( )
fun3( )
modules.py

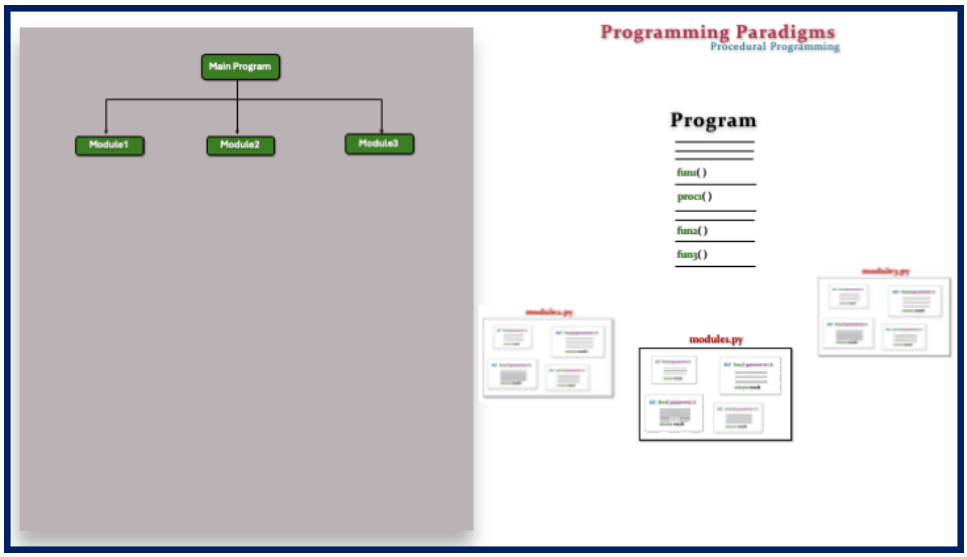

Programming Paradigms
Procedural Programming
Program
fun1( )
proc1( )
fun2( )
fun3( )
modules.py

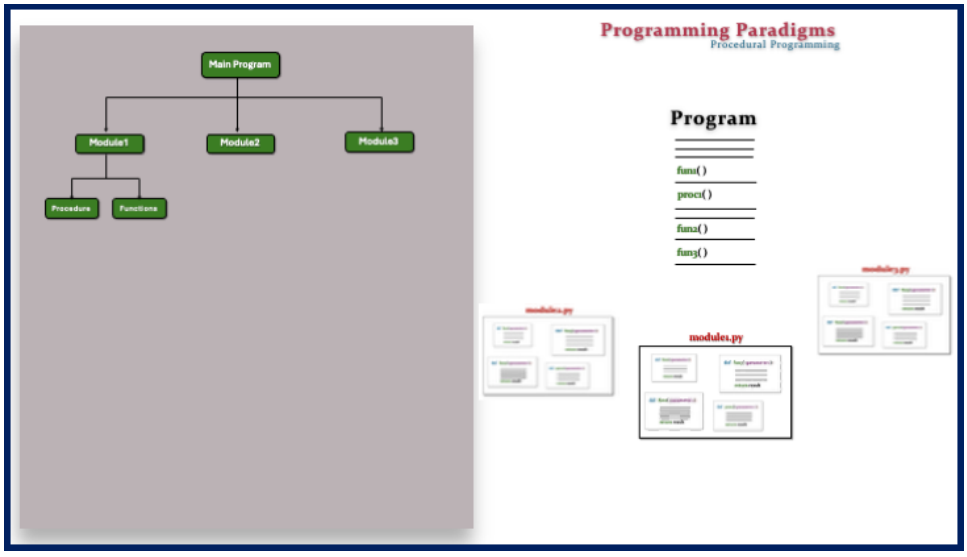

Programming Paradigms
Procedural Programming
Program
fun1( )
proc1( )
fun2( )
fun3( )
modules.py

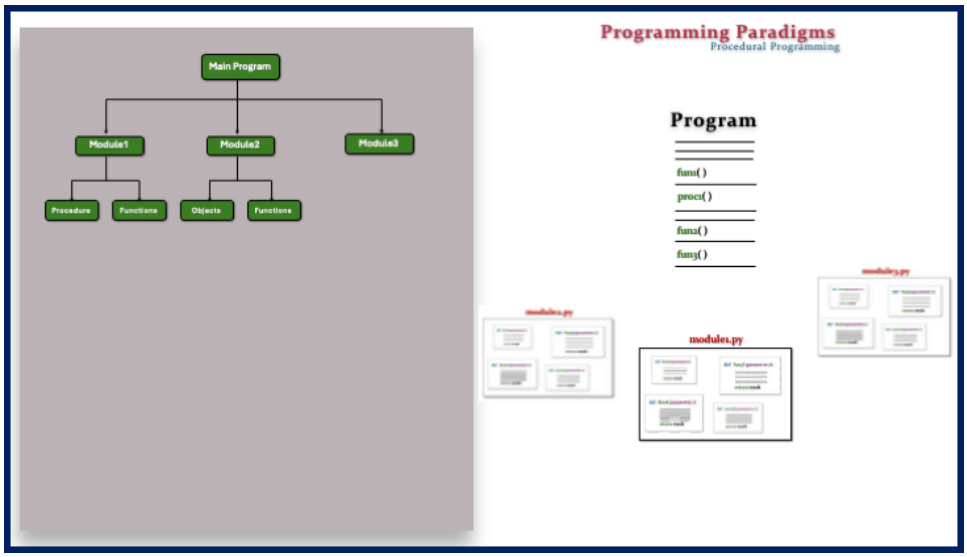

Main Program
Programming Paradigms
Procedural Programming
Program
fun1( )
proc1( )
fun2( )
fun3( )
modules.py

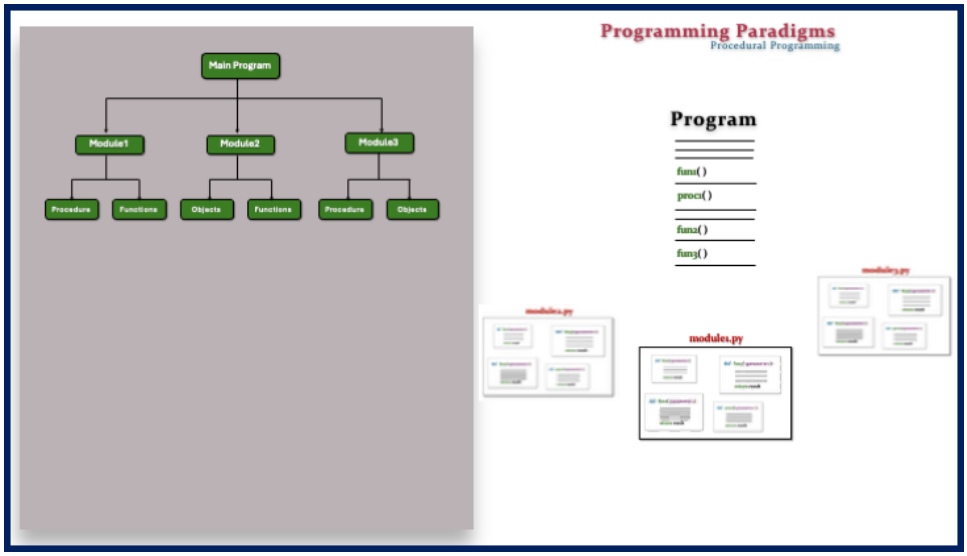

Main Program
Module1
Programming Paradigms
Procedural Programming
Program
fun1( )
proc1( )
fun2( )
fun3( )
modules.py

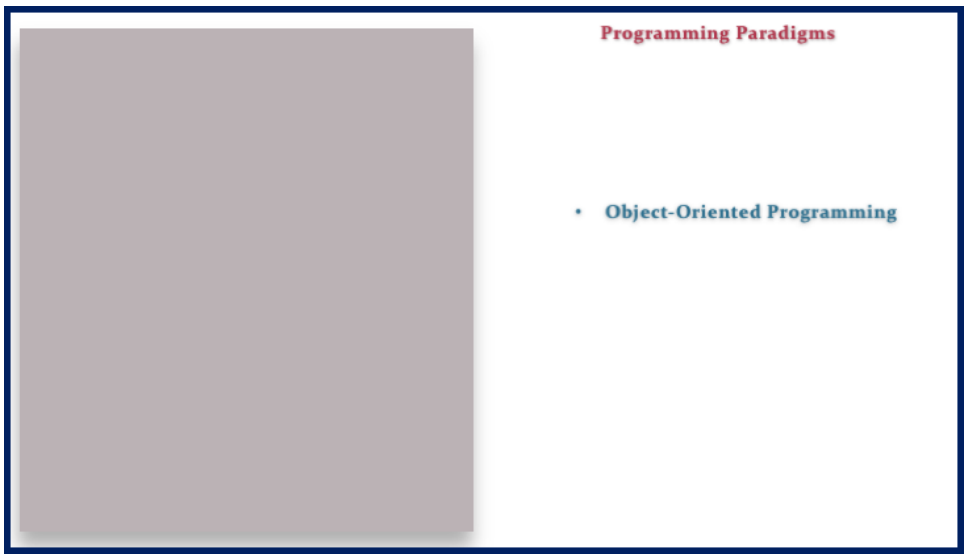

Main Program
Module1
Module2
Programming Paradigms
Procedural Programming
Program
fun1( )
proc1( )
fun2( )
fun3( )
modules.py

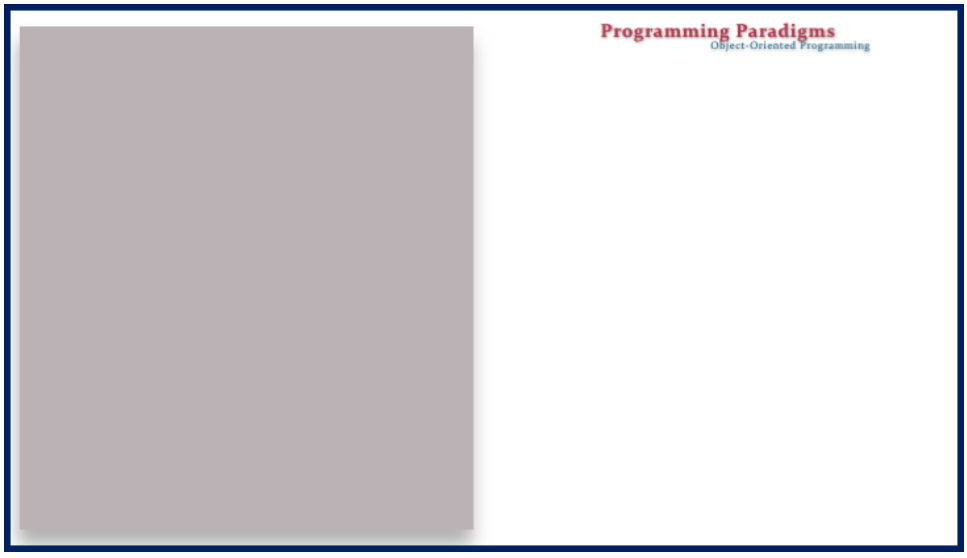

Main Program
Module1
Module2
Module3
Programming Paradigms
Procedural Programming
Program
fun1( )
proc1( )
fun2( )
fun3( )
modules.py

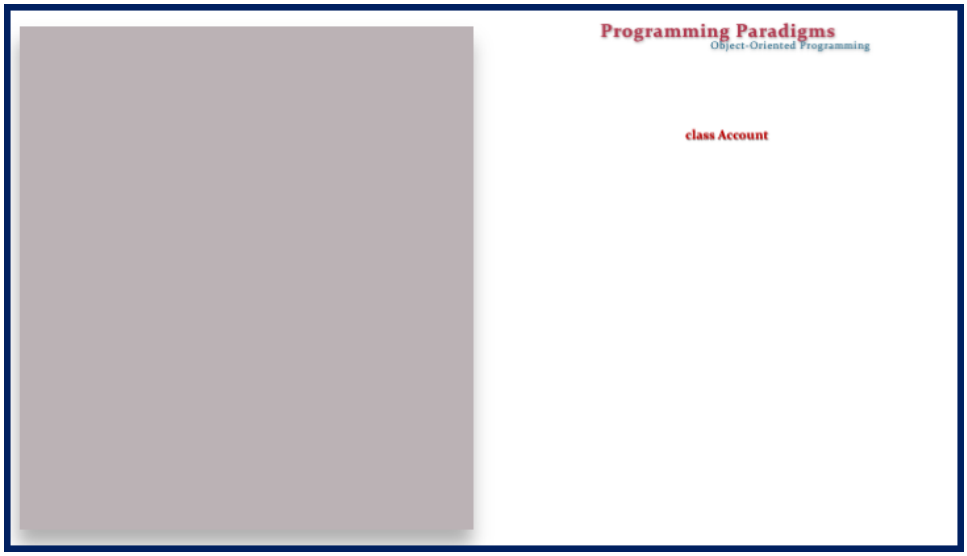

Main Program
Module1
Module2
Module3
Procedure
Functions
Programming Paradigms
Procedural Programming
Program
fun1( )
proc1( )
fun2( )
fun3( )
modules.py

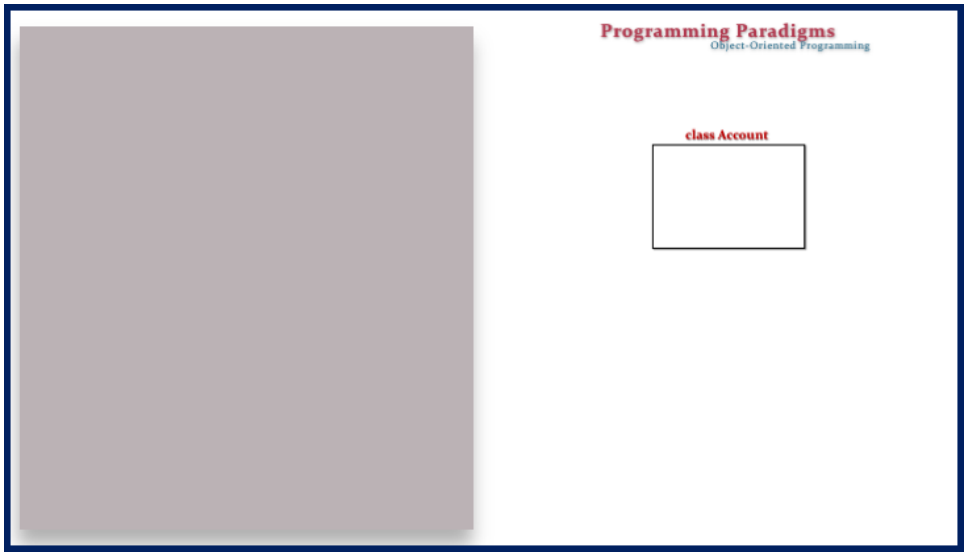

Main Program
Module1
Module2
Module3
Procedure
Functions
Objects
Functions
Programming Paradigms
Procedural Programming
Program
fun1( )
proc1( )
fun2( )
fun3( )
modules.py

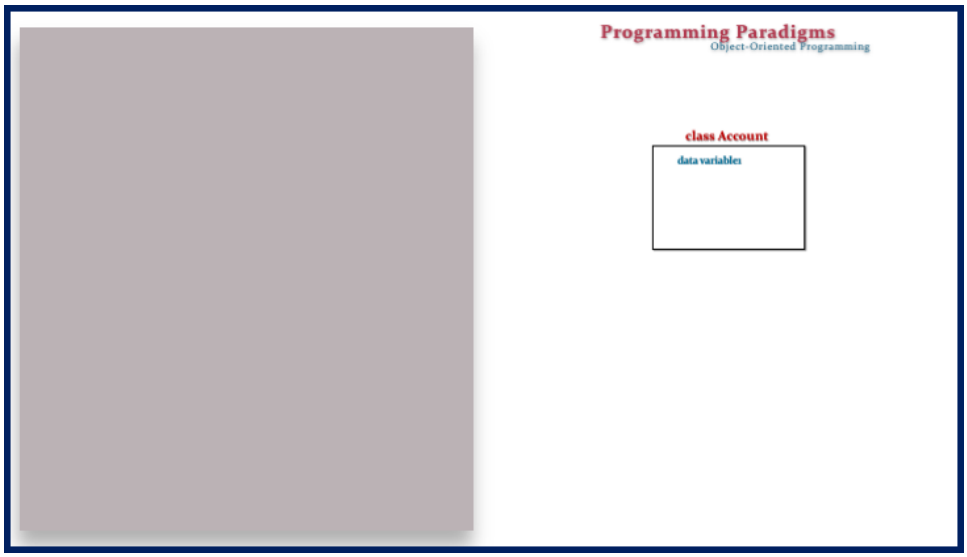

Main Program
Module1
Module2
Module3
Procedure
Functions
Objects
Functions
Procedure
Objects
Programming Paradigms
Procedural Programming
Program
fun1( )
proc1( )
fun2( )
fun3( )
modules.py

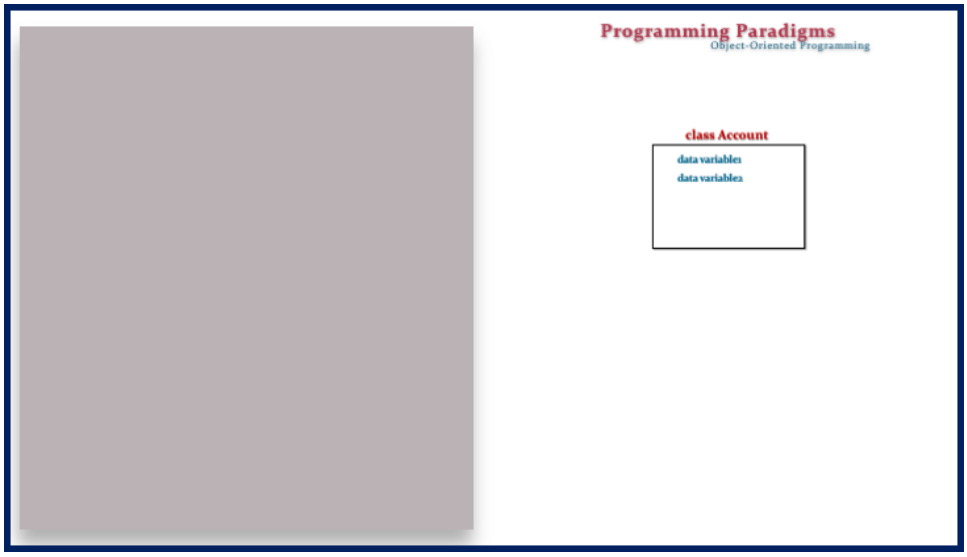

Programming Paradigms
• Object-Oriented Programming

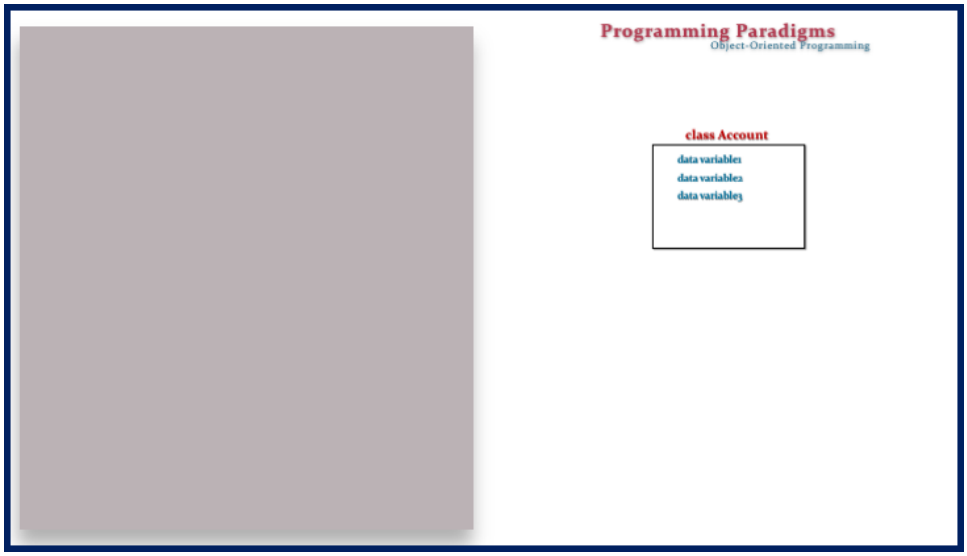

Programming Paradigms
Object-Oriented Programming

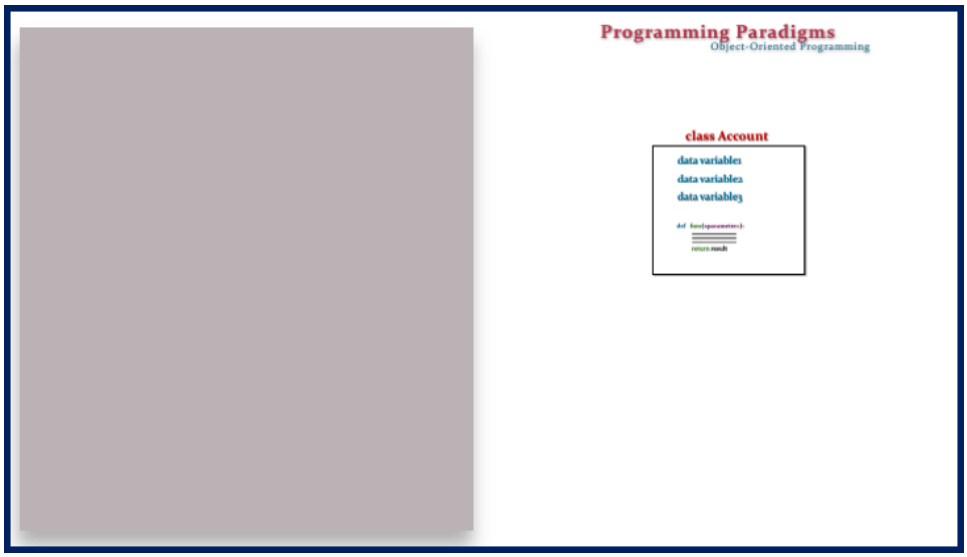

Programming Paradigms
Object-Oriented Programming
class Account

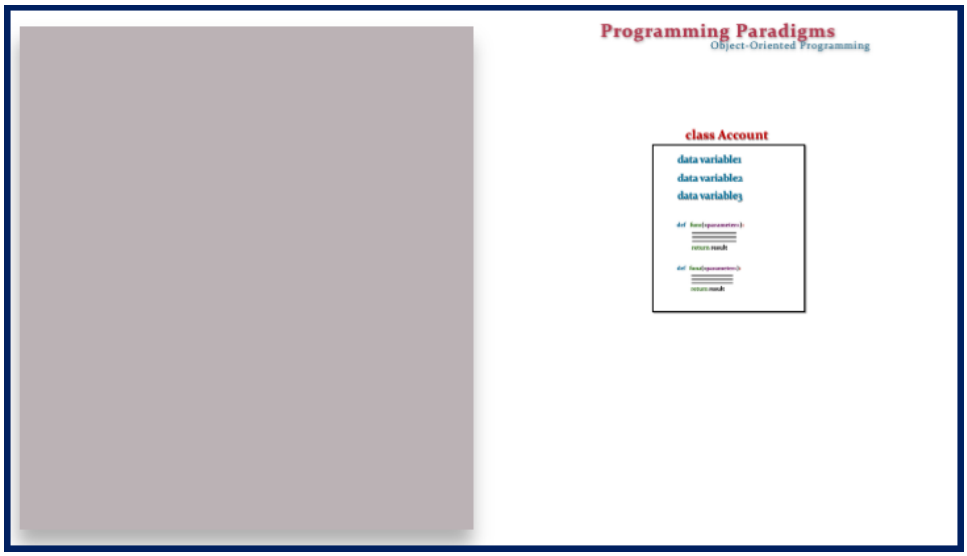

Programming Paradigms
Object-Oriented Programming
class Account

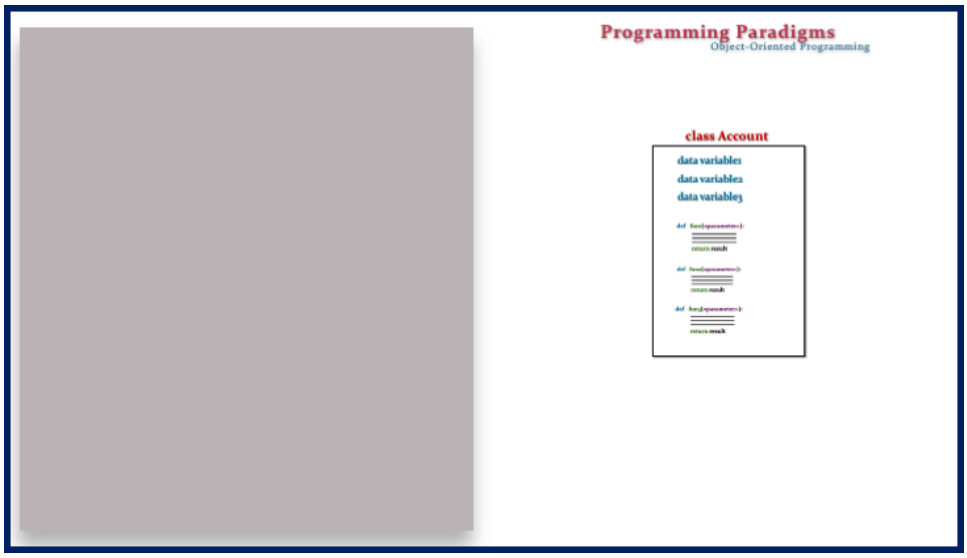

Programming Paradigms
Object-Oriented Programming
class Account
data variable1

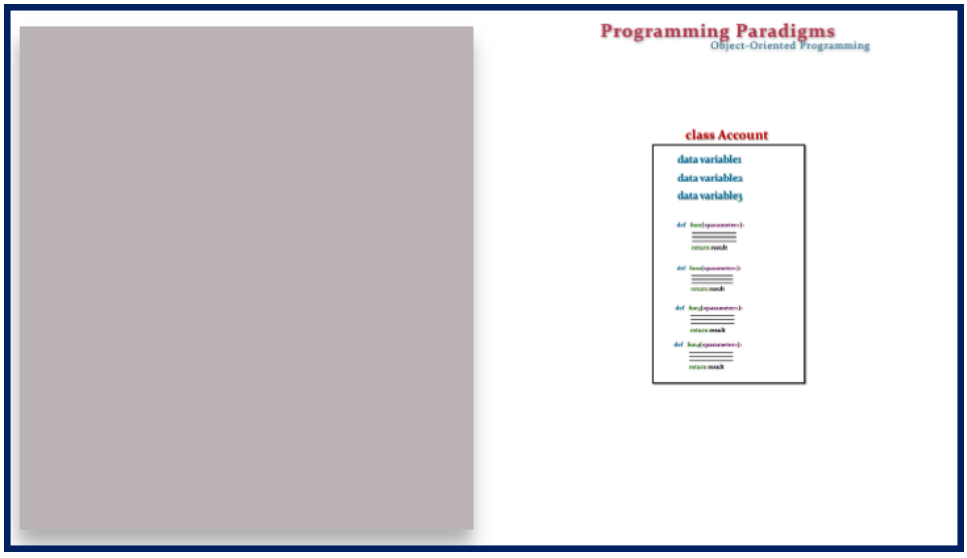

Programming Paradigms
Object-Oriented Programming
class Account
data variable1
data variable2

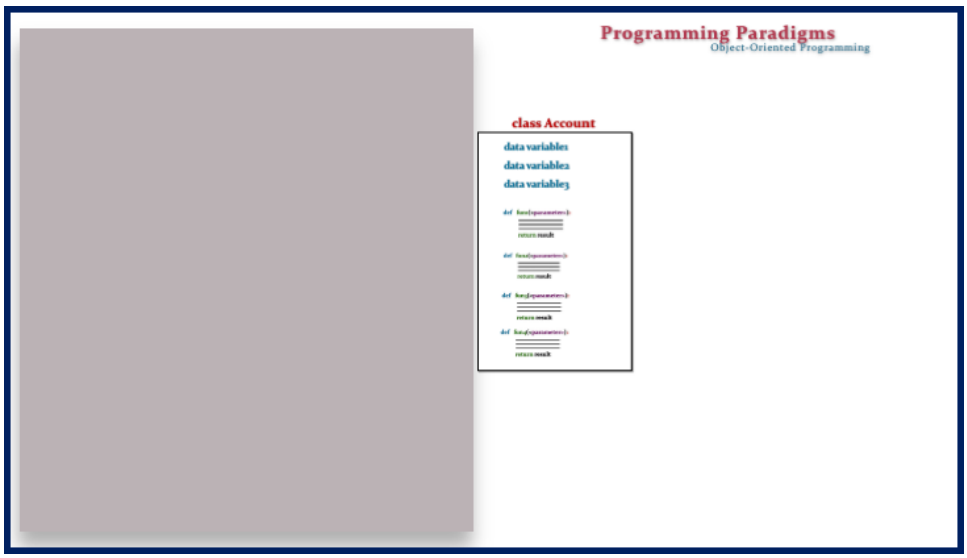

Programming Paradigms
Object-Oriented Programming
class Account
data variable1
data variable2
data variable3

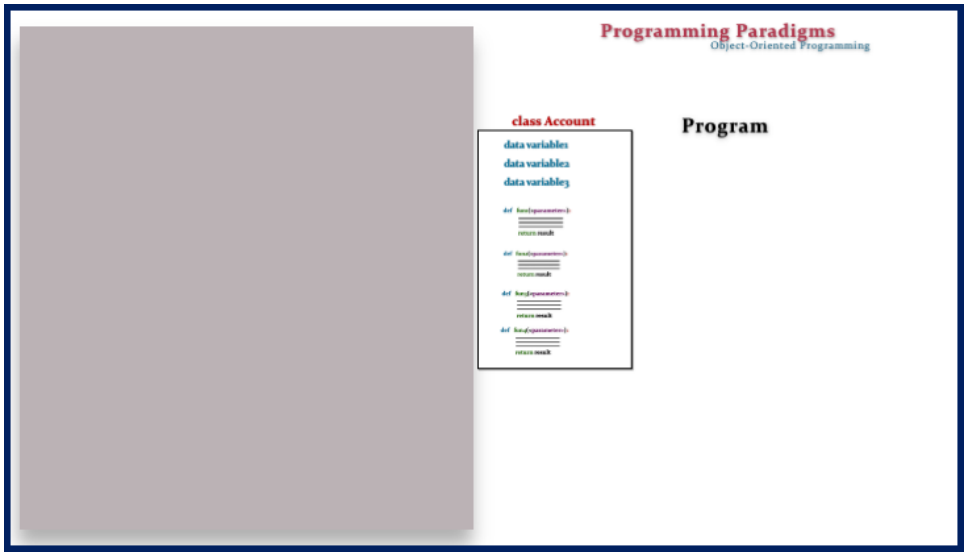

Programming Paradigms
Object-Oriented Programming
class Account
data variable1
data variable2
data variable3
def func(parameters):
return result

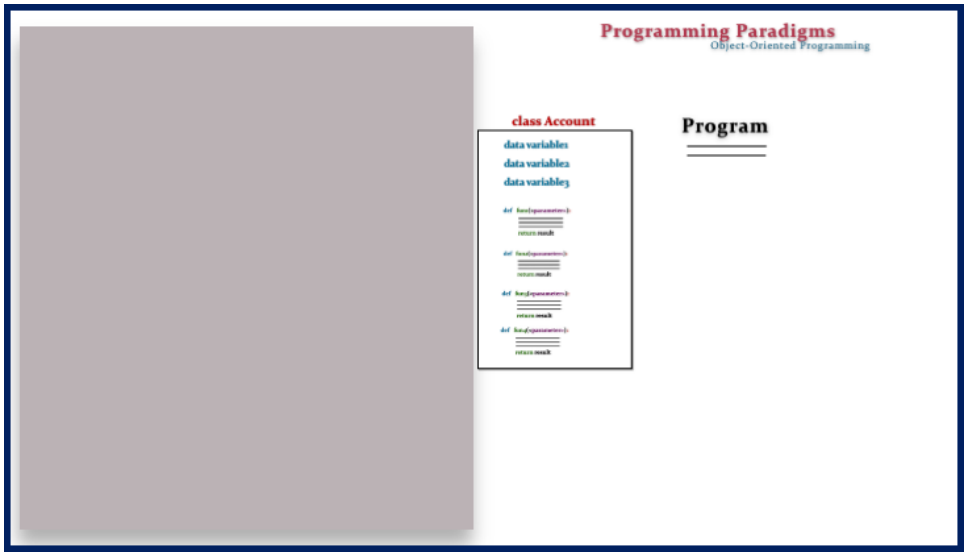

Programming Paradigms
Object-Oriented Programming
class Account
data variable1
data variable2
data variable3
def func(parameters):
return result
def func(parameters):
return result

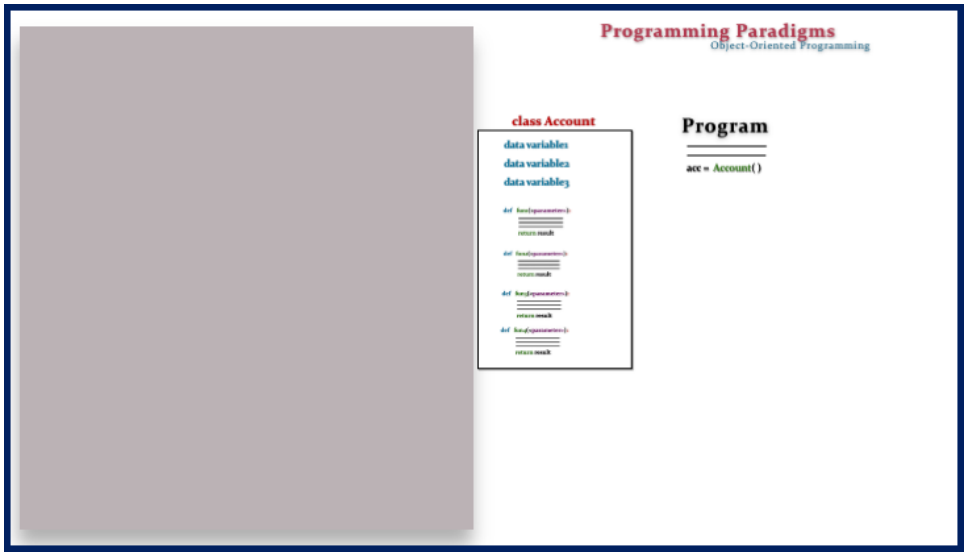

Programming Paradigms
Object-Oriented Programming
class Account
data variable1
data variable2
data variable3
def func(parameters):
return result
def func(parameters):
return result
def func(parameters):
return result

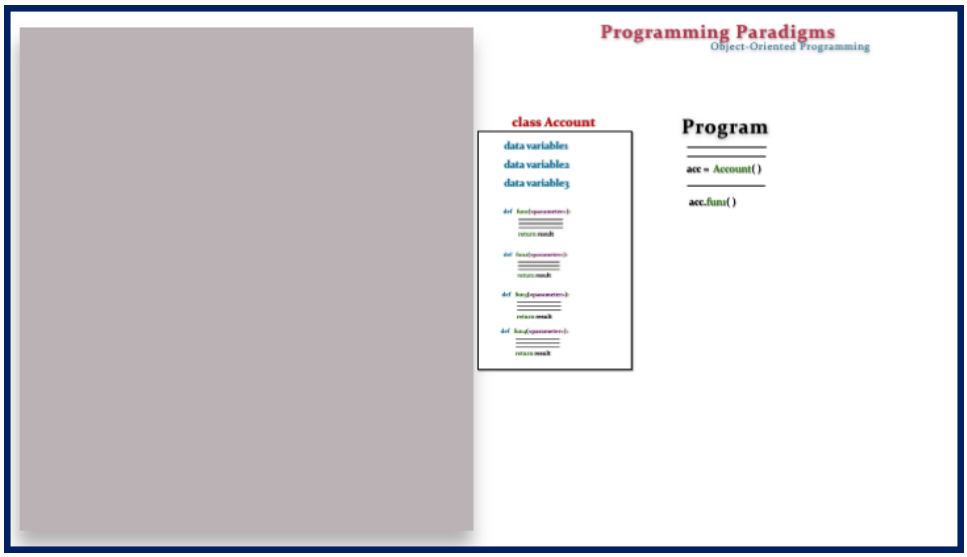

Programming Paradigms
Object-Oriented Programming
class Account
data variable1
data variable2
data variable3
def func(parameters):
return result
def func(parameters):
return result
def func(parameters):
return result
def func(parameters):
return result

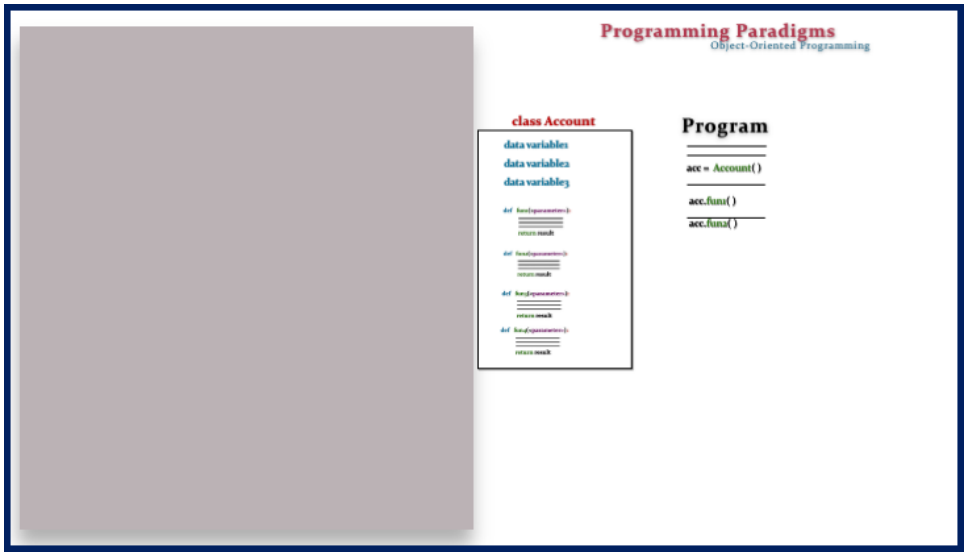

Programming Paradigms
Object-Oriented Programming
class Account
data variable1
data variable2
data variable3

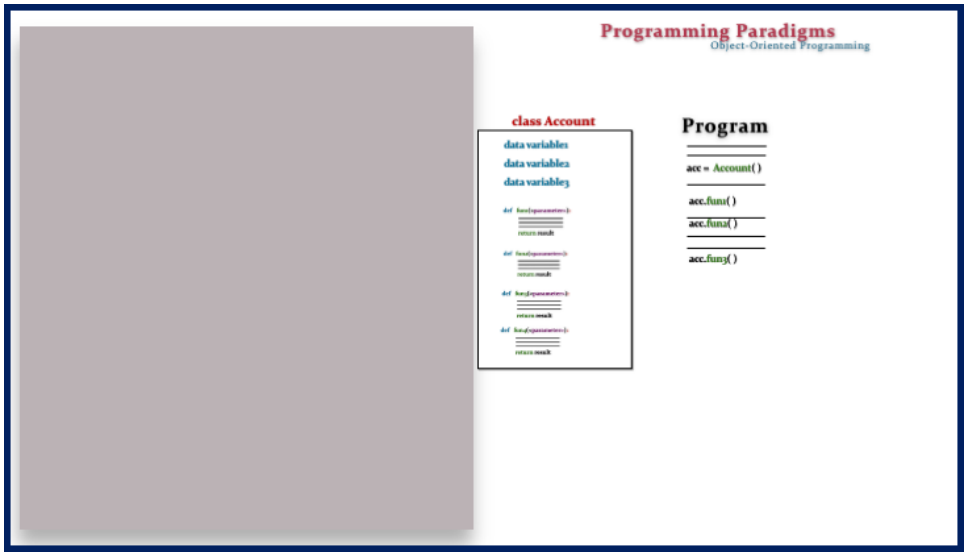

Programming Paradigms
Object-Oriented Programming
class Account
data variable1
data variable2
data variable3
Program

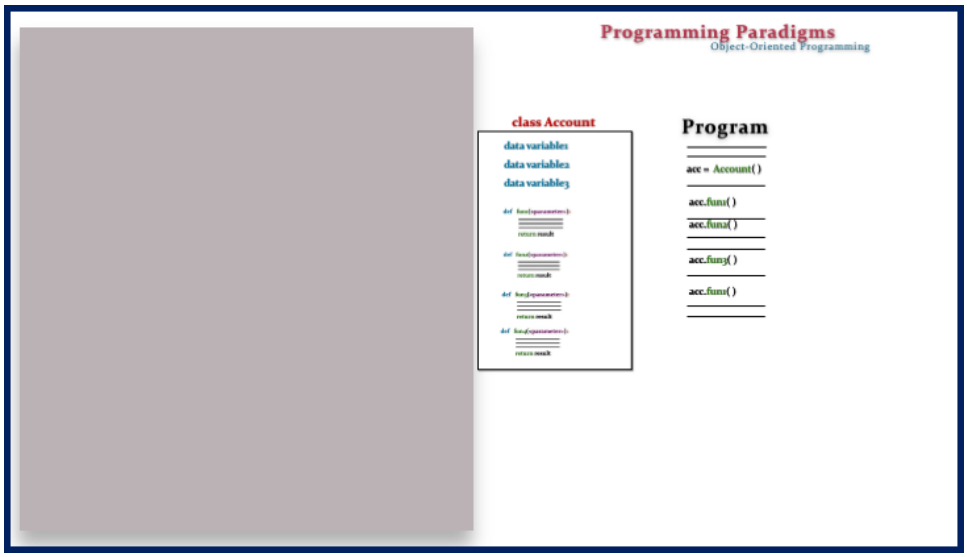

Programming Paradigms
Object-Oriented Programming
class Account
data variable1
data variable2
data variable3
Program

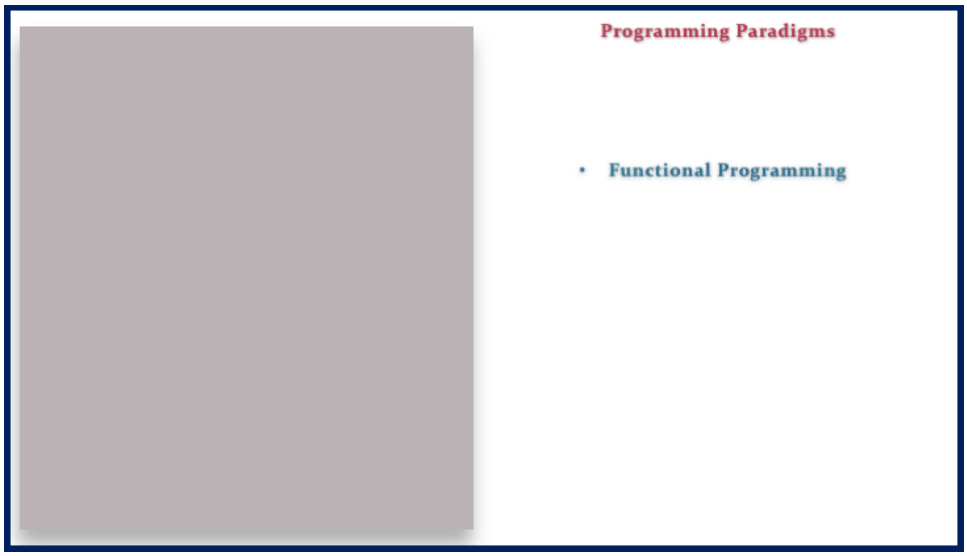

Programming Paradigms
Object-Oriented Programming
class Account
data variable1
data variable2
data variable3
Program
acc = Account( )

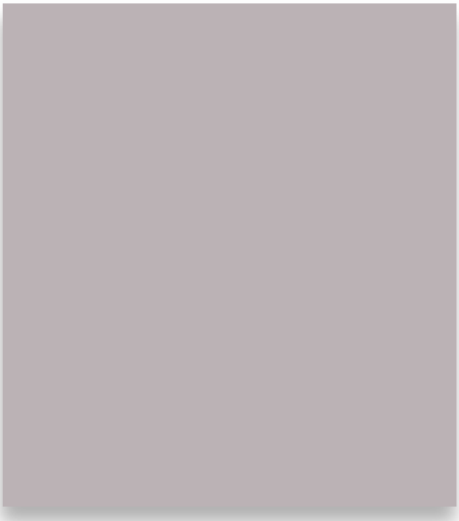

Programming Paradigms
Object-Oriented Programming
class Account
data variable1
data variable2
data variable3
Program
acc = Account( )
acc.fun1( )


Programming Paradigms
Object-Oriented Programming
class Account
data variable1
data variable2
data variable3
Program
acc = Account( )
acc.fun1( )
acc.fun2( )


Programming Paradigms
Object-Oriented Programming
class Account
data variable1
data variable2
data variable3
Program
acc = Account( )
acc.fun1( )
acc.fun2( )
acc.fun3( )


Programming Paradigms
Object-Oriented Programming
class Account
data variable1
data variable2
data variable3
Program
acc = Account( )
acc.fun1( )
acc.fun2( )
acc.fun3( )
acc.fun1( )


Programming Paradigms
• Functional Programming



Programming Paradigms

- **Functional Programming**
  - Mathematical Functions